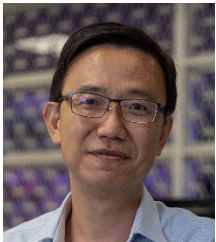


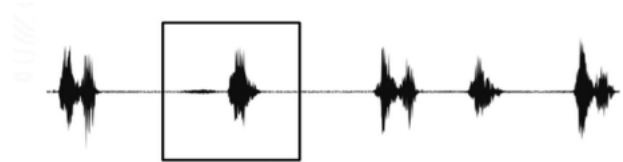


SC4172 Internet of Things - TinyML

Wake Word Detection (Chapter 7 and 8 TinyML Book)



Oh Hong Lye
Senior Lecturer
SCSE, Nanyang Technological University.
Email: hloh@ntu.edu.sg



Keyword (Wake-word) Detection

- One of the most famous and widespread examples of TinyML in the real world is the use of **wake words to wake up digital assistant** in mobile phones or smart home devices. Examples:
 - “**Alexa**”, “**Hey Google**”, “**Hey Siri**”.
- Two-Stage System
 - **On Device**
 - A **tiny, highly quantized neural network**
 - listens 24/7 for the wake word (e.g., "Hey Siri").
 - **Cloud**
 - When wake word is detected, the device records the command ("...what is the weather?") and sends it to the cloud (or a large on-device model) to understand the complex sentence.
 - It also re-analyse the wake word to prevent false alarm.

Why On-Device?

- Privacy

- If implemented in the cloud, the device would have to stream every single word spoken in your house to the server 24/7 to check if you said the wake word. This is a massive privacy violation.
- By doing it locally, the audio never leaves the device until after the trigger is confirmed.

- Battery Life

- Keeping the Wi-Fi/LTE radio active to stream audio constantly would drain a smartphone battery in a couple of hours.
- The microcontroller is designed to run for days/months.

- Latency

- Excessive delay will be incurred when sending data back and forth the server if model is in the cloud.
- Impact on user experience

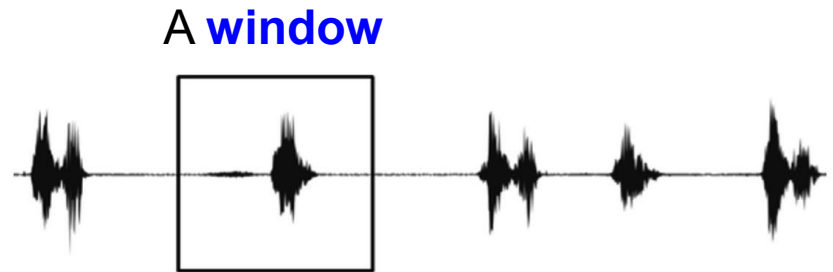


FEATURE ENGINEERING

Feature Engineering

- Feature engineering
 - transforming **raw sensor data** into **usable information**
 - make it **easier** life is for the AI algorithms to attempt **interpreting** it.
- When working with sensor data that involves **time series data**
 - Feature engineering involves **chopping** the **data** into manageable **chunks** – known as **windows**
 - Typically makes use of **digital signal processing algorithms** to **condition and/or transform** the input data

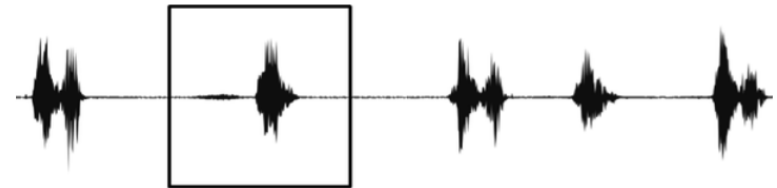
- E.g. Keyword detection



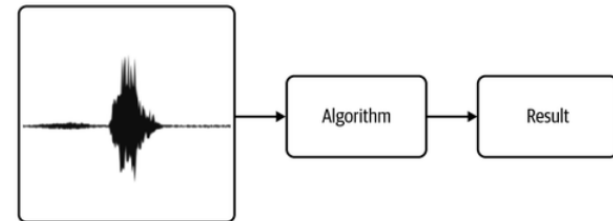
Source: AI at the Edge, Daniel Situnayake and Jenny Plunkett

Time series data

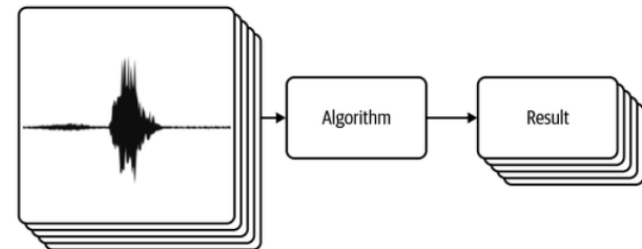
- Time series data is often broken into chunks of **windows** to be **analysed** one at a time
- **Latency** is the time taken to **process** one window
- **Stride** is the **time step** we move forward to create the next window
- **Frame rate** is the **number of windows** that can be processed **per second**
- Frames could be **sequential** or **overlapped** with each other.



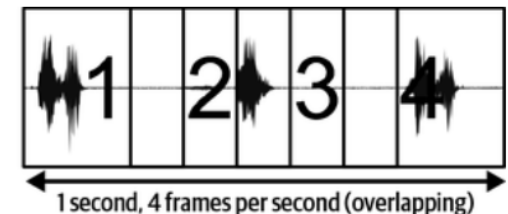
1. A window is taken from a stream of data.



2. The window is processed by an algorithm.



3. A series of windows produces a series of results.



Source: AI at the Edge, Daniel Situnayake and Jenny Plunkett

Latency and Frame Rate

Good to have trigger points for sampling but may not always be available.

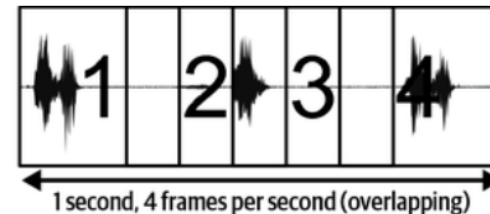
- The **lower** the **latency**
 - the **higher** the **max frame rate**
 - **more windows** of data can be analysed in a given period of time.
 - **more analysis** lead to **more reliable** the results
- Latency affects **how close the Windows can be taken**
 - **Lower latency** allows **shorter window separation**
 - **Shorter** window separation (or overlap windows) allow more data to be captured.
 - **Longer** window separation might result in missing **critical** part of the signal



Source: AI at the Edge, Daniel Situnayake and Jenny Plunkett

Window Size

- Latency is directly proportional to Window size.
- Larger windows
 - contain more information about the signal
 - longer time to process the data within it (Latency increases)
 - high-fidelity data and potentially better accuracy
- Overlap windows
 - Occur when $\text{Stride} < \text{Window Size}$
 - E.g. If Stride is 200ms and Window is 1000ms, the new window overlaps the previous one by 800ms.

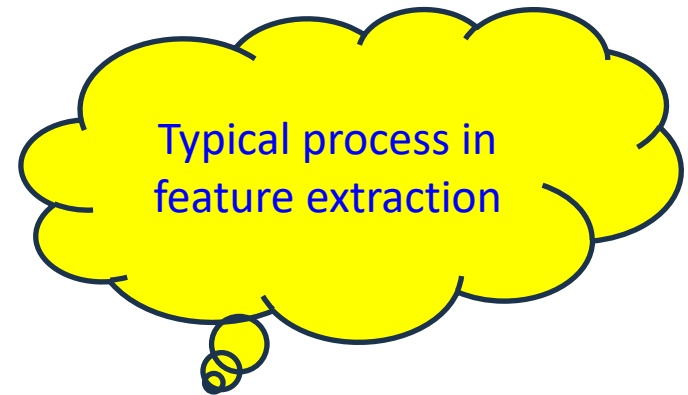


Summary

- Window size determine Latency
 - Increasing Window Size always increases Latency
 - Lower latency implies higher Max Frame Rate
- Stride (or Overlap) determine the demand.
 - Demand implies the number of frames the application needs to process per second
 - As Stride increases, the number of frames per second that needs to be processed decreases
- To meet real-time requirement
 - Processing speed needs to be faster than demand,
 - i.e. Latency < Stride
- Larger window and shorter strides enables more data to be captured but increases the computation requirement.

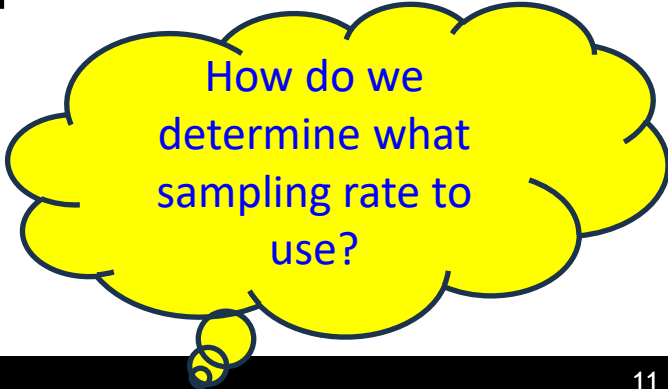
Signal Conditioning and extraction

- Feature engineering typically makes use of digital signal processing algorithms
 - Resampling
 - Filtering
 - Spectral analysis
 - Feature detection



Resampling

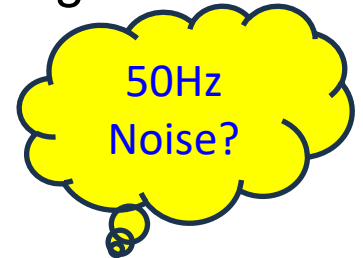
- Time series signals are **sampled** at a **certain rate/frequency**
 - in terms of the number of data samples per second (Hz)
 - often necessary to **change** the sample rate of a signal
- **Up sampling**
 - **increase** the sampling rate of a signal by **inserting addition data** in between sample)
 - E.g. to enable a signal to be analysed alongside another signal that has a higher sampling rate.
- **Down sampling**
 - **throw away** some of the sampled **data**, similarly, to **match** signals with lower sampling rate for analysis purpose



How do we
determine what
sampling rate to
use?

Filtering

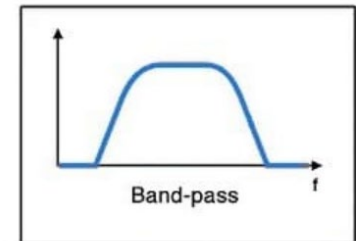
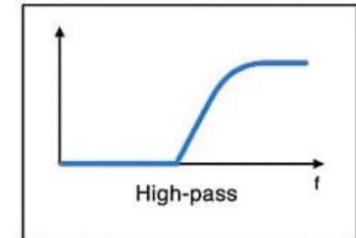
- All signals from **sensors** contain some level of **noise**, this could be **random** fluctuations in the data that happen due to slight inaccuracies in measurement
 - Examples:
 - **speckles** in a digital camera photograph taken at night
 - **background hum** in audio recordings
- If the noise is present at **specific frequencies**
 - **filters** can be used to remove
- **Filtering** is a very common signal processing operation
 - many embedded processors provide **hardware support** for some types of filtering
 - Multiply-and-Accumulate (**MAC**) unit
 - **reduces latency** and **power** consumption.



Type of Filters

- Low-pass filters
 - allow frequencies **below** the **cutoff** frequency to pass through and attenuating those above.
 - Example of a common Lowpass digital filter is the moving average filter
- High-pass filters
 - allow frequencies **above** the **cutoff** frequency to pass through and attenuating those below.
- Band-pass filters
 - allow frequencies **within** a **certain** frequency **band** to pass through and attenuating those outside the band.

Low Pass, High Pass and Band Pass Filters



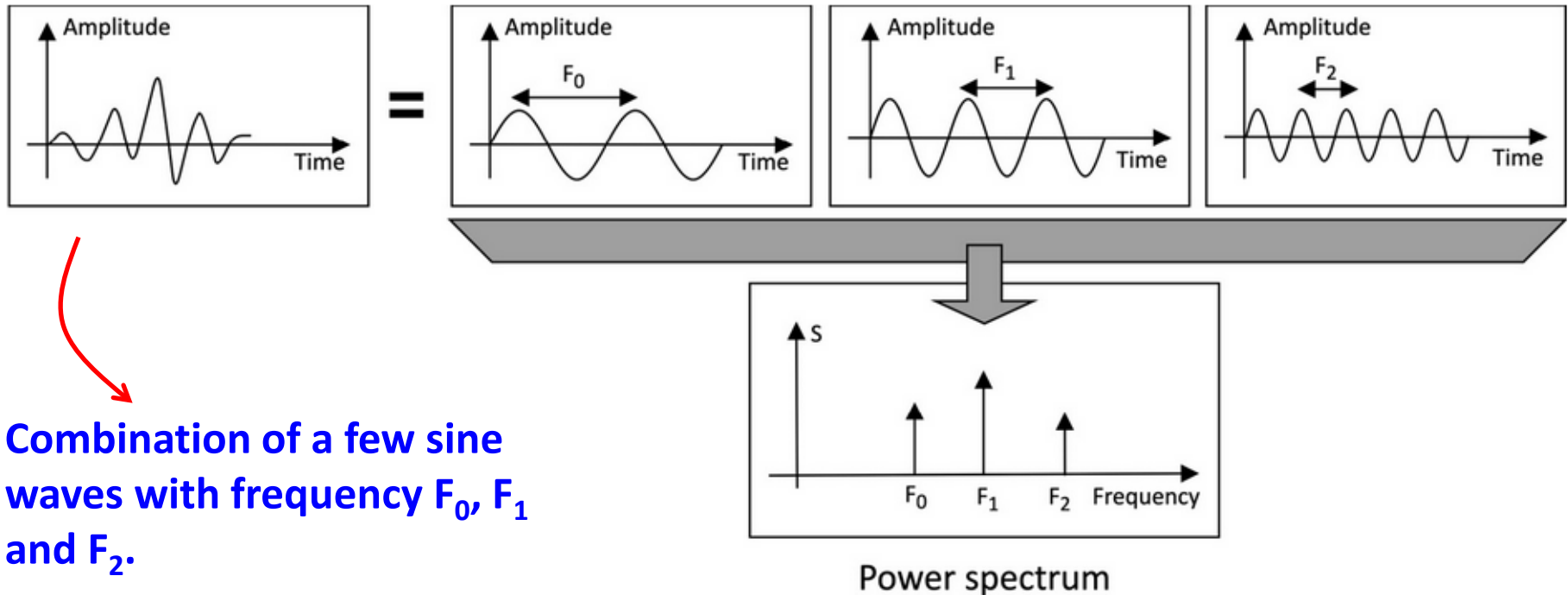
Source: rfpag.com

Spectral Analysis (Frequency Domain)

- A time series signal
 - occur in time domain,
 - meaning it represents how a set of variables change over time.
- But it is common to transform a time series signal into the frequency domain
 - E.g. using Fourier transform
- More convenient to analyse values obtained through transformation
 - describe how much of the signal lies in various frequency bands over a range of frequencies i.e. a spectrum.

Power Spectrum

- Frequency Power Spectrum can be obtained by using Fourier Transform

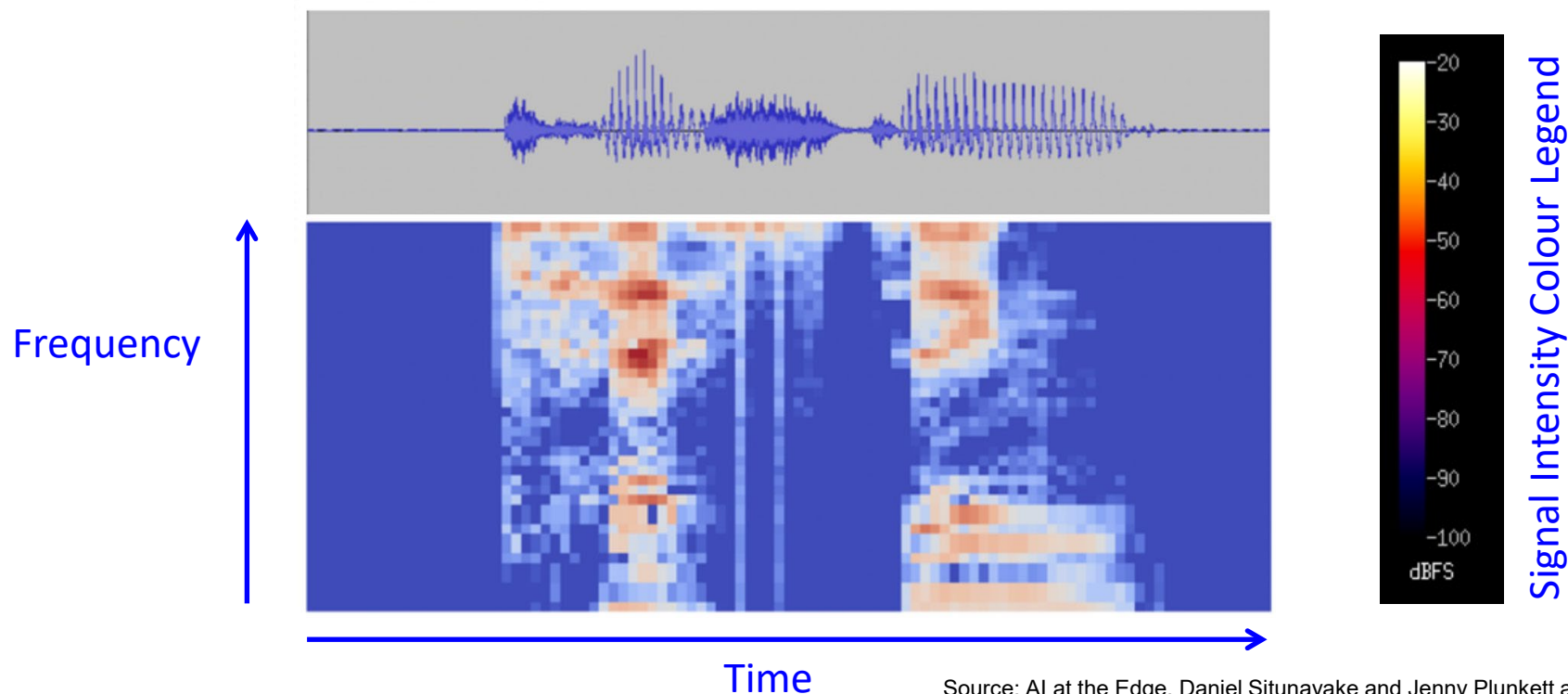


Source: TinyML Cookbook, Gian Marco Iodice, Ronan Naughton

Spectrogram

Does the
spectrogram look
like an image?

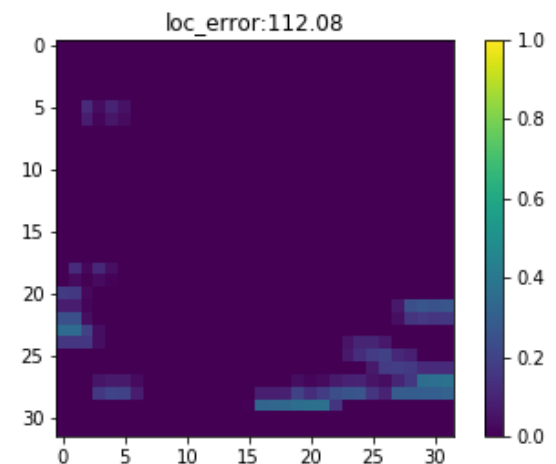
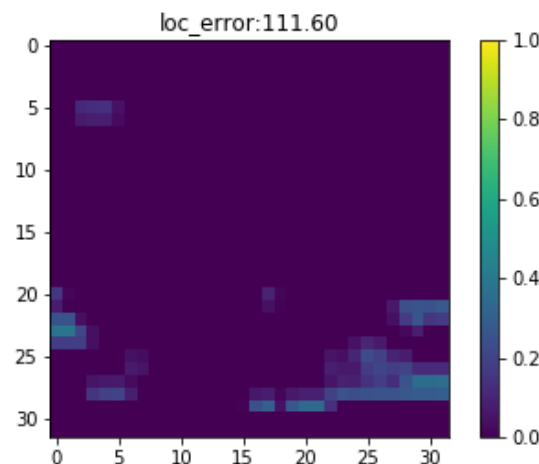
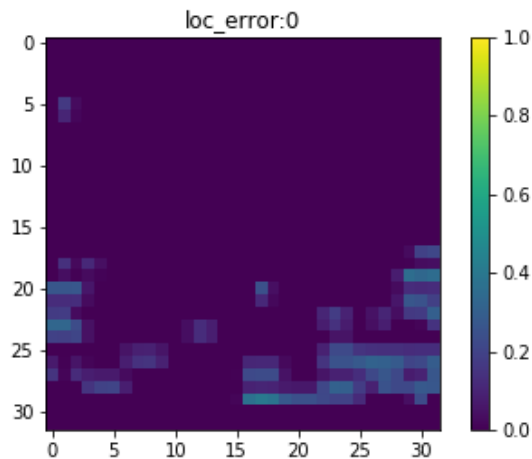
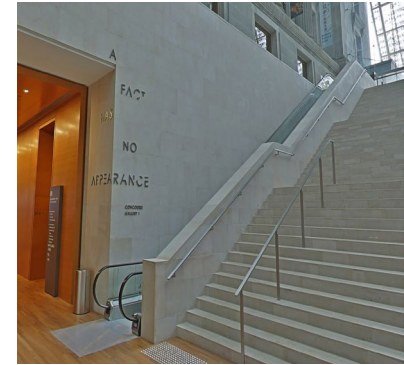
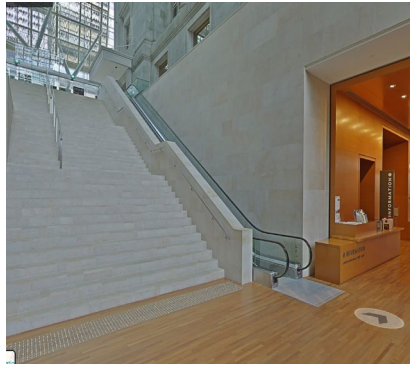
- Spectrogram
 - a **map** of how the signal's **frequencies change over time**
 - serves as a very effective input to machine learning models
 - commonly used in **audio application**.



Source: AI at the Edge, Daniel Situnayake and Jenny Plunkett and Wikipedia.

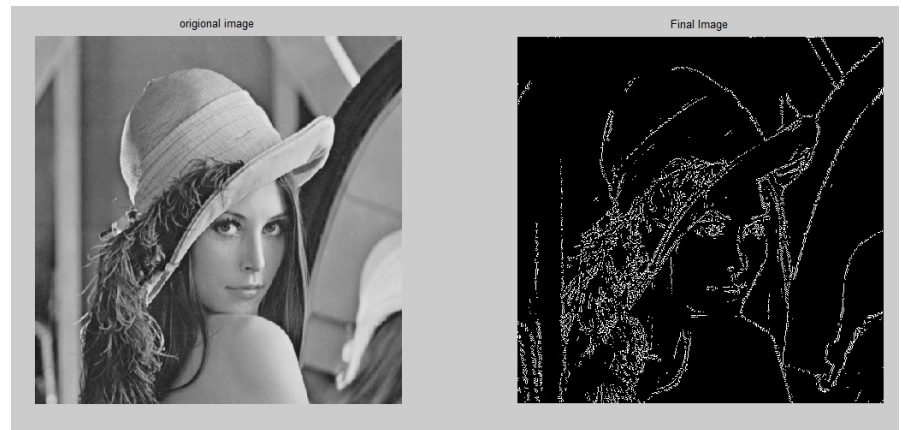
More 'Spectrogram': Wifi RSSI Heat Map

- **RSSI**: Receiver Signal Strength Indicator, i.e. the power intensity of the detected Wifi signal.



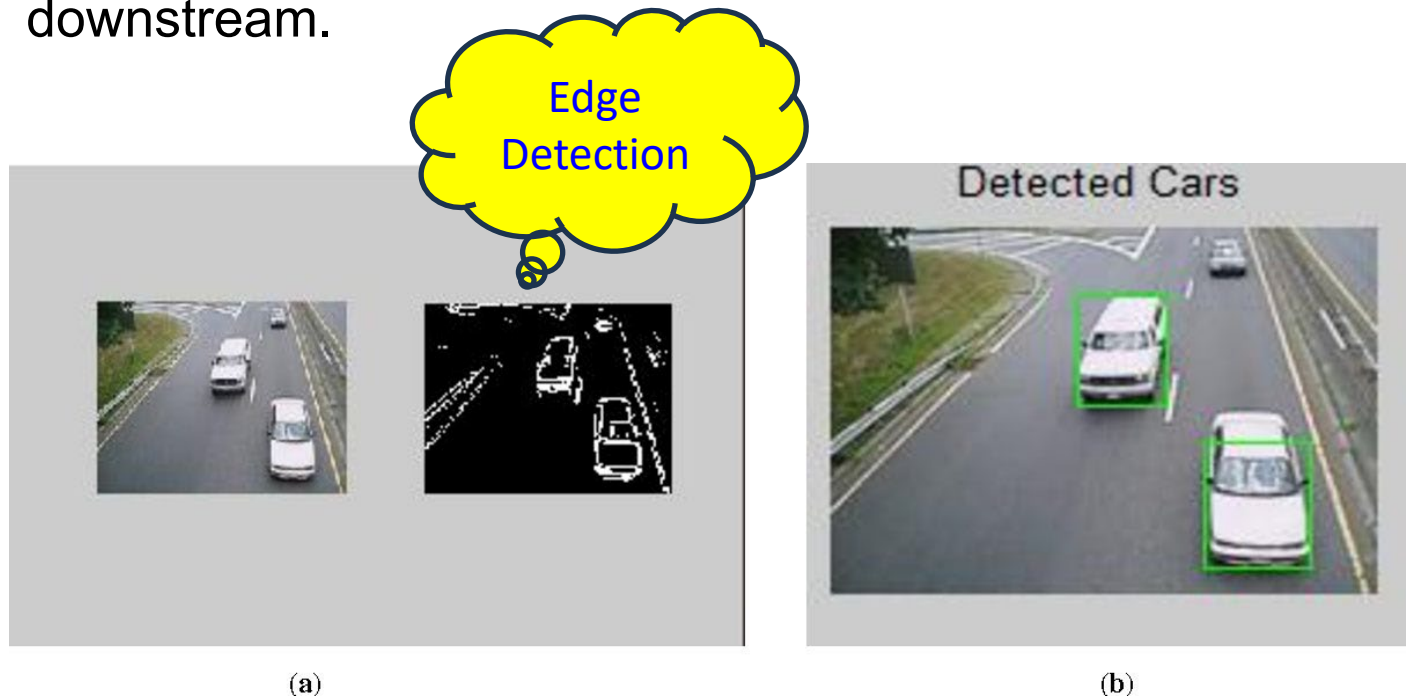
Feature Detection for Image

- There are many signal processing algorithms dealing with the **extraction** of **useful features** from **images**
 - computer vision algorithms (E.g. OpenCV)
- For microcontroller - **OpenMV**
 - an open-source library of feature detection algorithm implementations along with hardware designed to run them.
- Examples:
 - **Edge Detection** used to identify boundaries in an image



Feature Detection for Image

- Image feature detection
 - reduces a big, messy image into a more compact representation of key visual structures that are present within it
 - facilitate for any subsequent AI algorithms operating downstream.

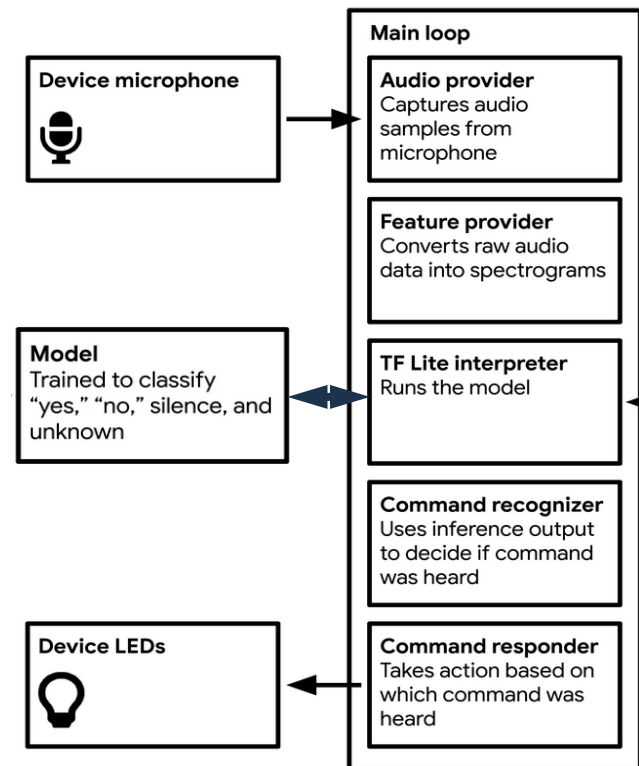


Source: Application of Edge Detection for Vehicle Detection in Traffic Surveillance System B. Ashwini, N. YuvarajuB

WAKE WORD DETECTION

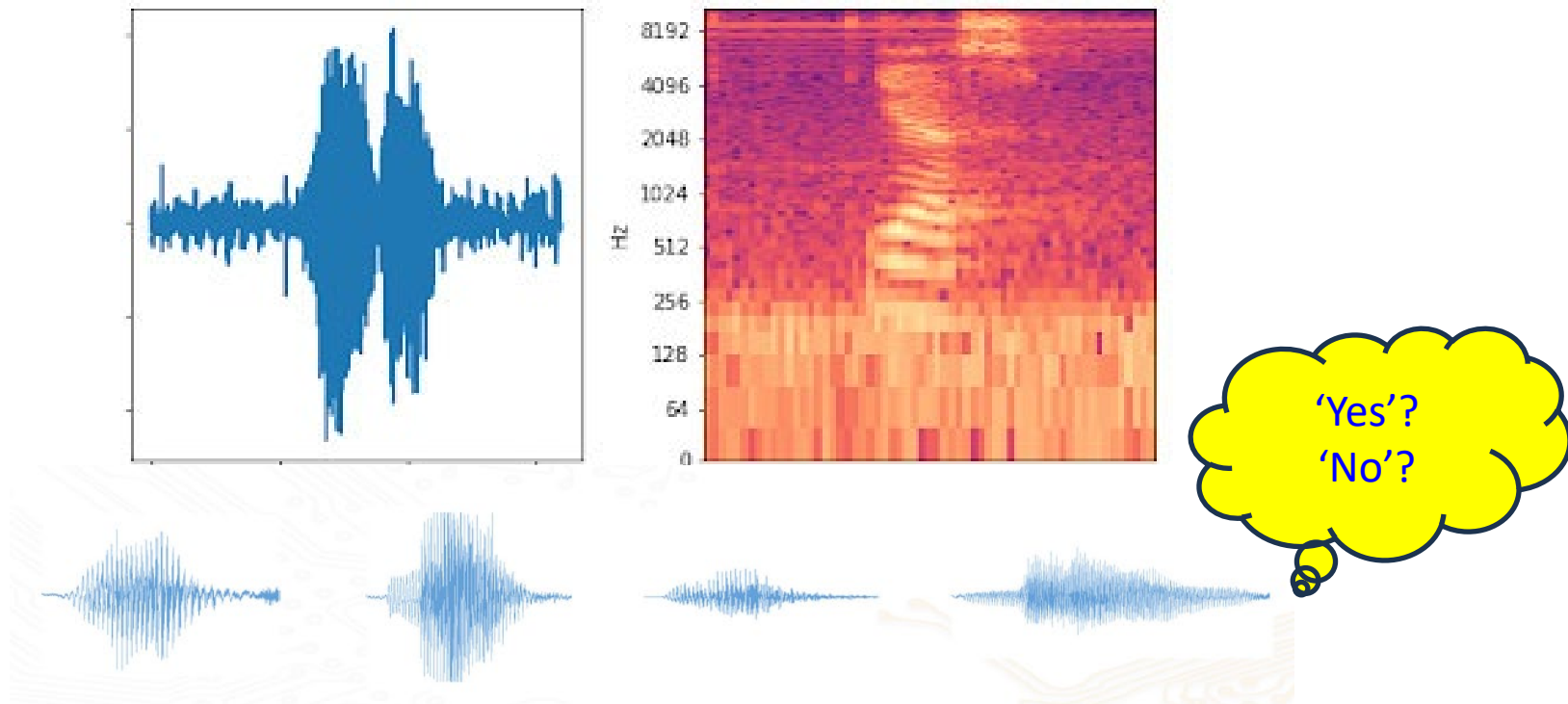
Key Word Detection Processing

- Obtains **audio data** as input
 - pre-processing to **extract feature** suitable to feed into a model
- Run **inference**
 - with continuous live stream time series data
- Model performs **classification**
 - output class probabilities
- **Recognise command**
- **Execute operation**



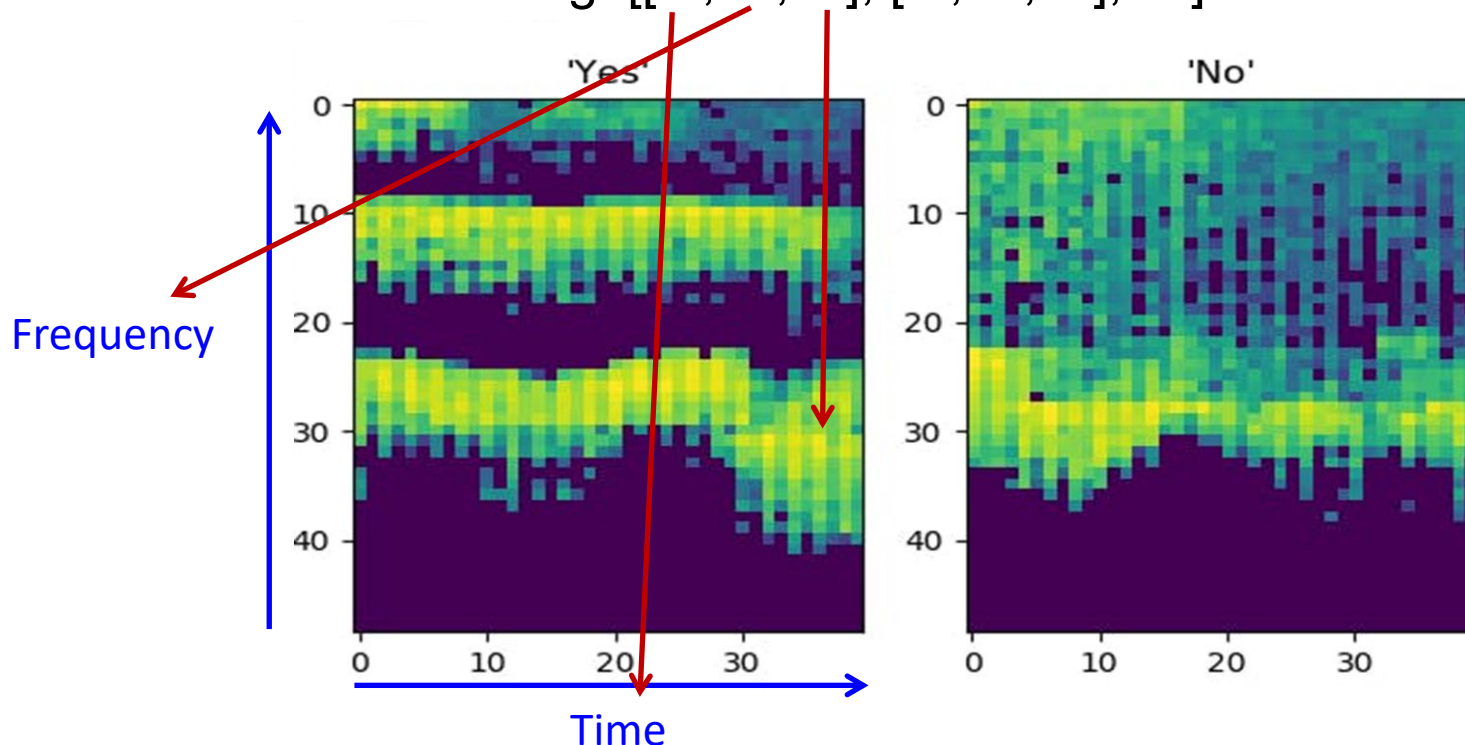
Audio pre-processing

- Raw audio speech
 - hard to identify its unique pattern in time domain (i.e. feature)
- Pre-processing to extract feature suitable for a model
 - in the form of spectrogram in frequency domain



Spectrogram

- Presenting the **sound visually** as an **image**
 - with a **time** and a **frequency** axis
 - pixel **intensities** representing the amplitude or energy of the sound at a specific time instance and frequency
 - **2D Tensors** e.g. $[[t_0, f_0, i_0], [t_0, f_1, i_1], \dots]$



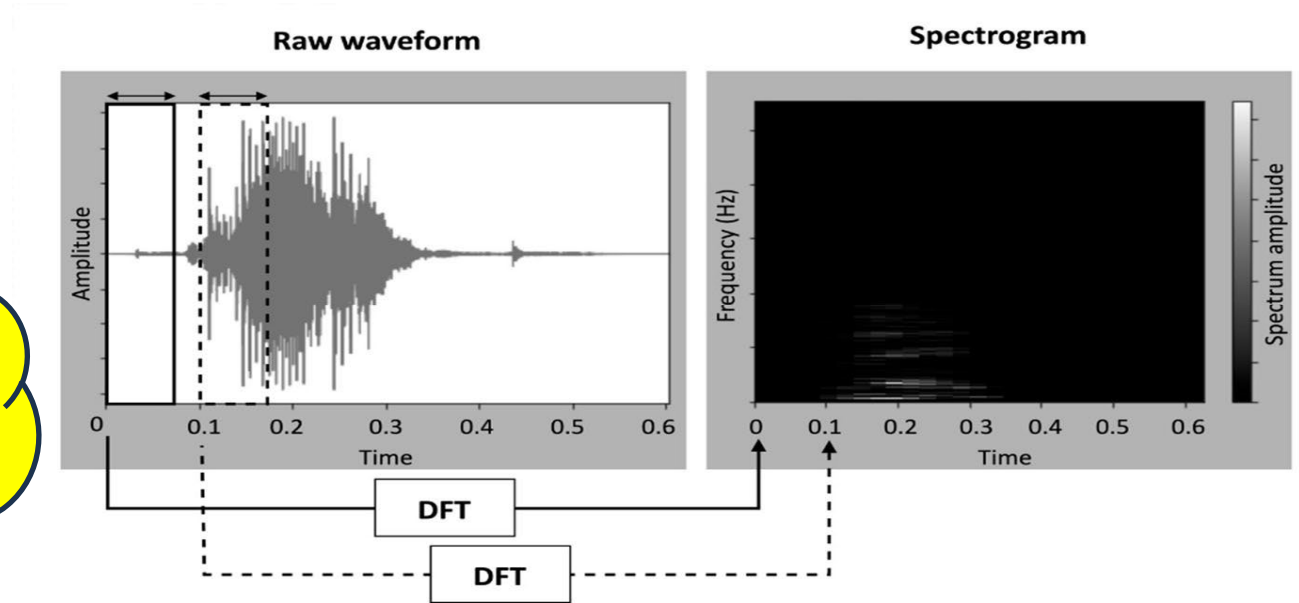
Spectrogram

- Advantage of using spectrograms over raw time domain wave data
 - an image classification problem
 - can be done using well know CNN based Image classifiers
- CNN
 - first detect simple feature like lines and edges
 - integrate to form more complex features
 - combine to form an image



Voice Feature Extraction

- A **spectrogram** for voice speech recognition (obtained by Discrete Fourier Transform - **DFT**)
 - **not effective** because the **relevant features** are **not emphasized**
 - the spectrogram is **dark** in almost all regions.



What can be done to make the data more effective?

Source: TinyML Cookbook, Gian Marco Iodice, Ronan Naughton

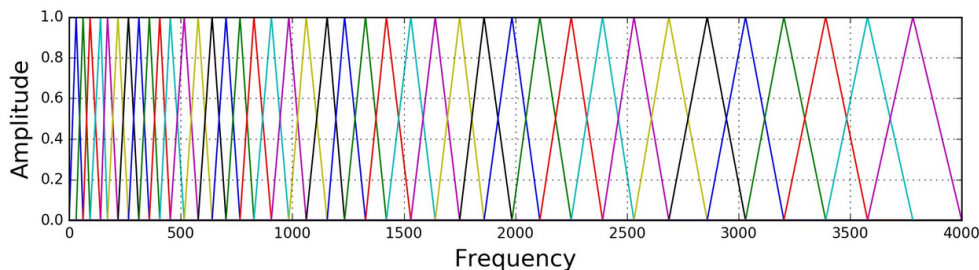
Human Perception of Sound Intensity

- Humans have far better hearing near 1000 Hz than at low frequencies like 50 Hz.
 - a 1000 Hz tone will seem far louder than the 50 Hz tone for most people even though the sound levels are identical.
- Most people are more sensitive to sounds with frequencies between 300 Hz to 3KHz.
- Speech recognition
 - spectrogram is adjusted to suit humans perceived frequencies and loudness
 - MFCC (Mel-frequency cepstral coefficients). Coefficients used to represent the spectral characteristics of sound in a way that is well-suited for various machine learning tasks
 - More details in next slide ...

Mel-frequency cepstral coefficients (MFCC)

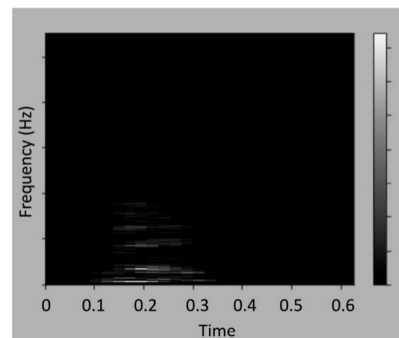
- The Mel scale is widely considered the "gold standard" feature extraction technique for speech recognition.
- Converts raw audio waveforms into a compact representation that mimics how the human ear perceives sound
 - Scaling the frequency in Hz to Mel with the Mel Scale filter bank to cater for non-linear aspect of human auditory system.
 - logarithmic (dB) scale rather than linear to mimic human loudness perception.
- Discrete Cosine Transform (DCT) used to compress the spectrogram by retaining the first 2-13 coefficients (cepstral coefficients) and discarding the rest. Deriving the MFCC.

Mel Scale Filter Bank

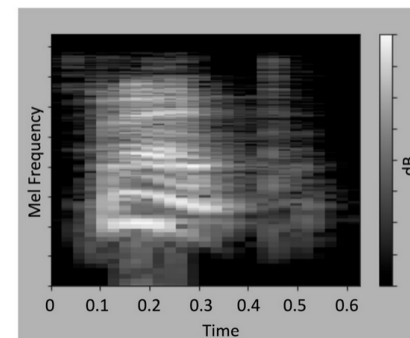


Note the increasing bandwidth of each bank as frequency increases

Spectrogram



Mel Spectrogram



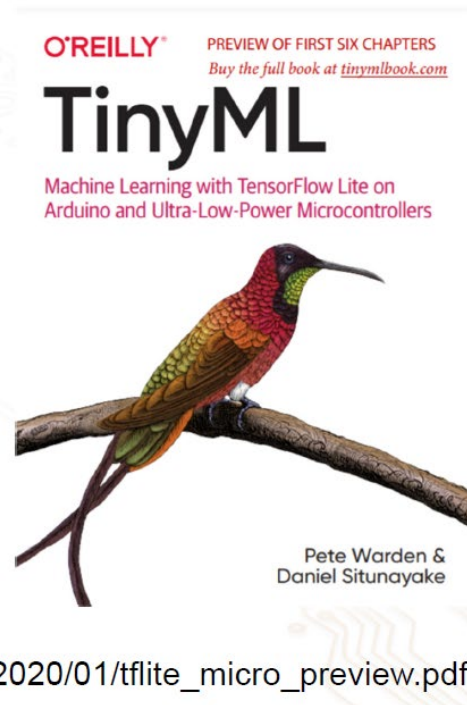
Spectrogram of 'red' utterance

Source: TinyML Cookbook, Gian Marco Iodice, Ronan Naughton

DEPLOYMENT

Reference

Chapter 7, 8 (& Appendix B)
TinyML Book

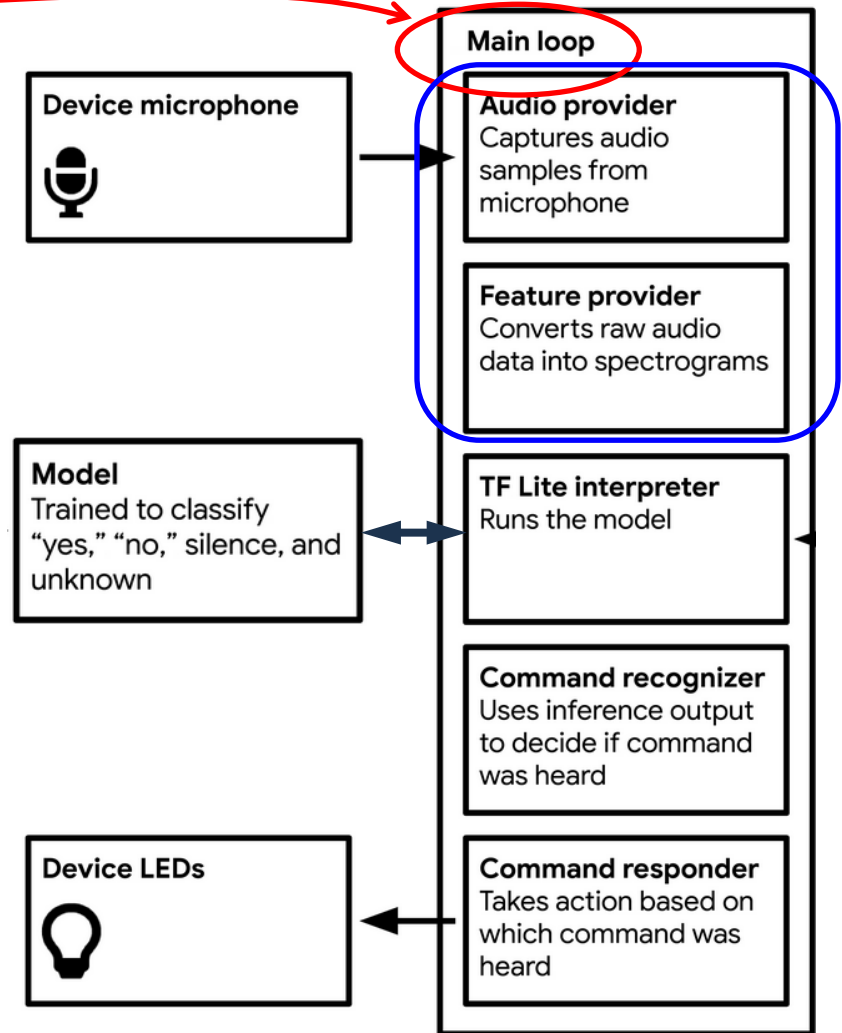


https://tinymmlbook.files.wordpress.com/2020/01/tflite_micro_preview.pdf

- Chap 7 and 8 are not in the preview pdf. It is available online via NTU Library site.
- Note also that you will find some difference in the code discussed in the book vs the main branch TFLite code in github.
- Links to the specific branch containing TFLite examples the author used can be found in the book.

TinyML Keyword Detection Implementation

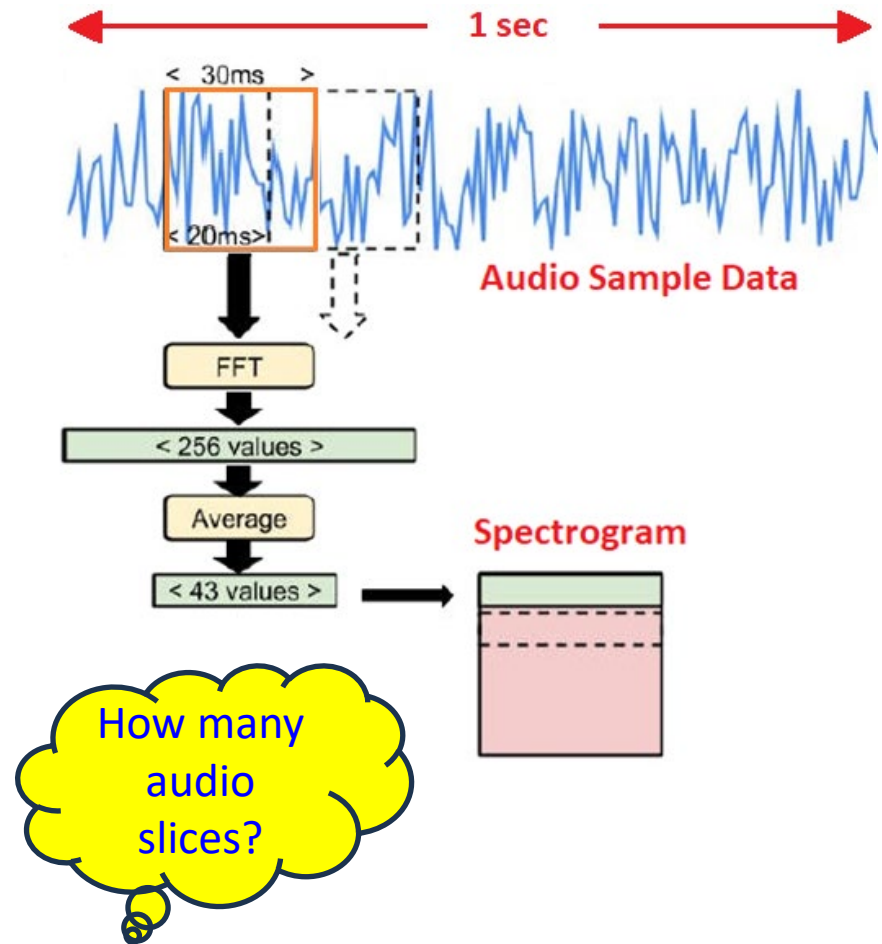
- **Main loop**
 - application run in a continuous loop executing the processes contained within the loop
- **Audio provider**
 - capture raw audio data from microphone (using callback when new data is available)
- **Feature Provider**
 - converts raw audio data into spectrogram format



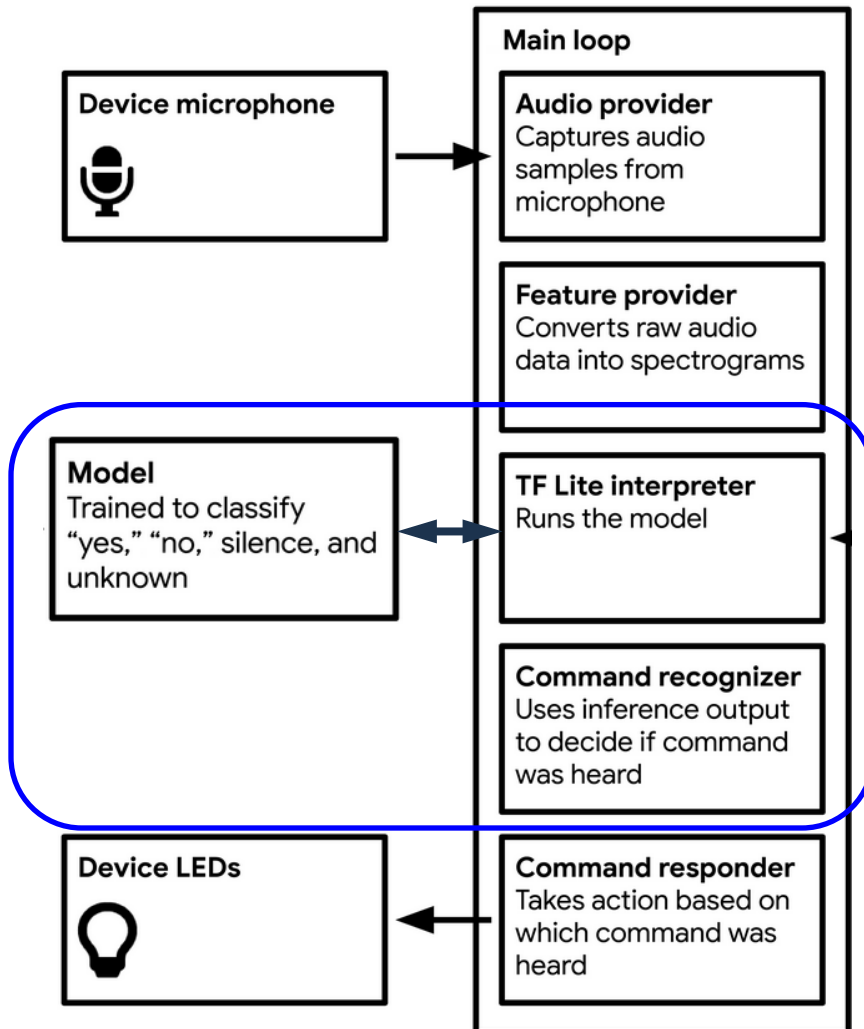
Source: TinyML Pete Warden & Daniel Situnayake

Feature Provider

- Collect **one second** of **raw audio** data.
- Perform conversion
 - by running a **30-ms slice** of audio through a Fast Fourier Transform (**FFT**) algorithm
 - each slice **overlapping** the previous slice by **10ms**
- Note that the features extracted must **match** with the **trained model**. If MFCC is used, then MFCC must be extracted, not just plain FFT.



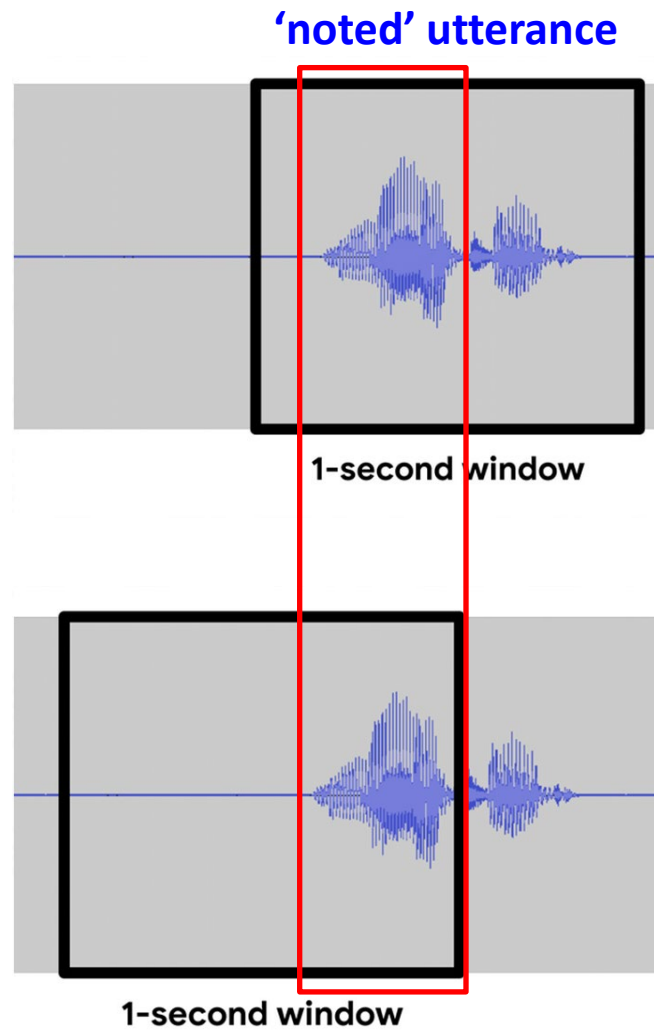
TinyML Keyword Detection Example



- **TF Lite interpreter**
 - run the TensorFlow Lite model, transforming the input spectrogram into a set of probabilities
- **Model**
 - a data array trained to recognised 'Yes', 'No', (and unknown)
- **Command recognizer**
 - determine whether a known word was recognized

Source: TinyML Pete Warden & Daniel Situnayake

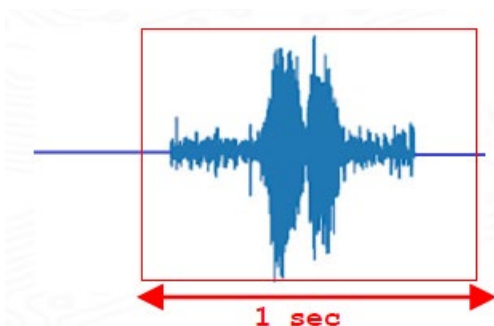
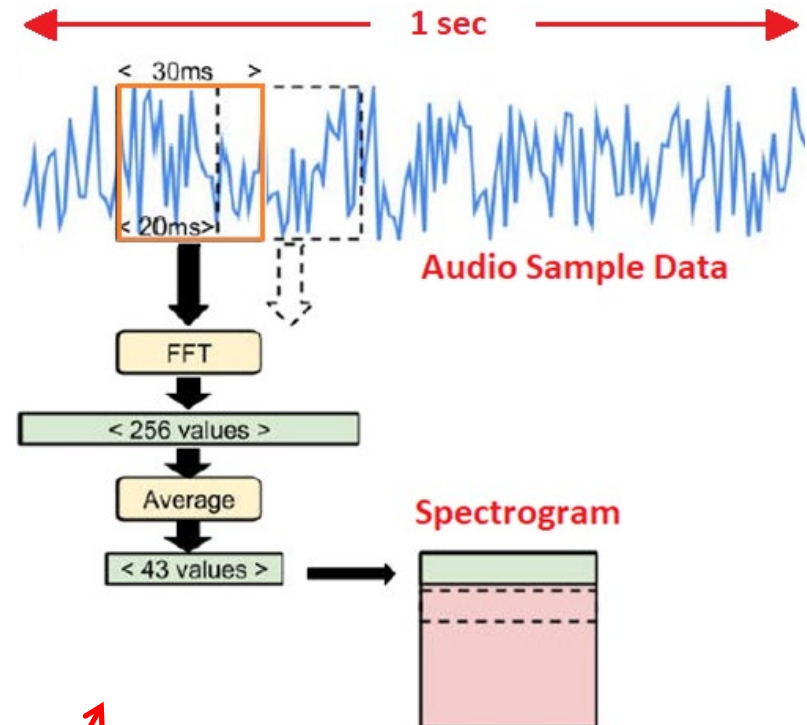
Recognizing an utterance



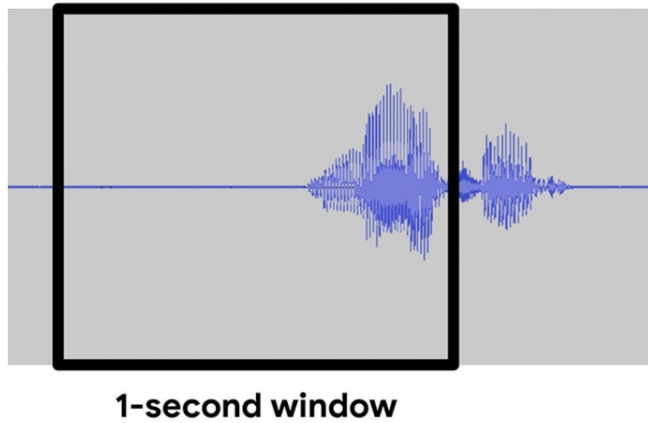
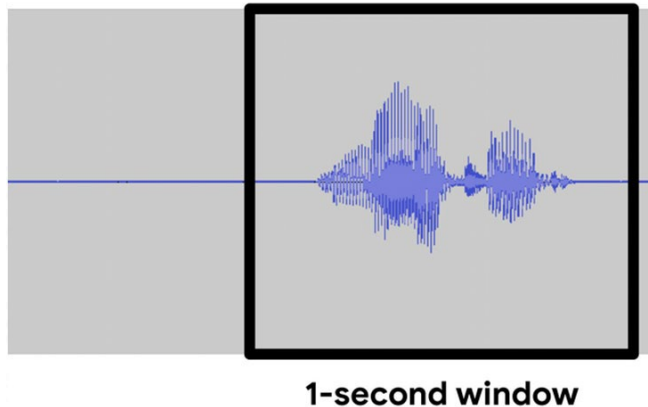
- More complex than it seemed because of the time series data.
- Audio data for the word 'noted'
- If only part of the data is captured
 - sounds like 'no'

Command Recognizer

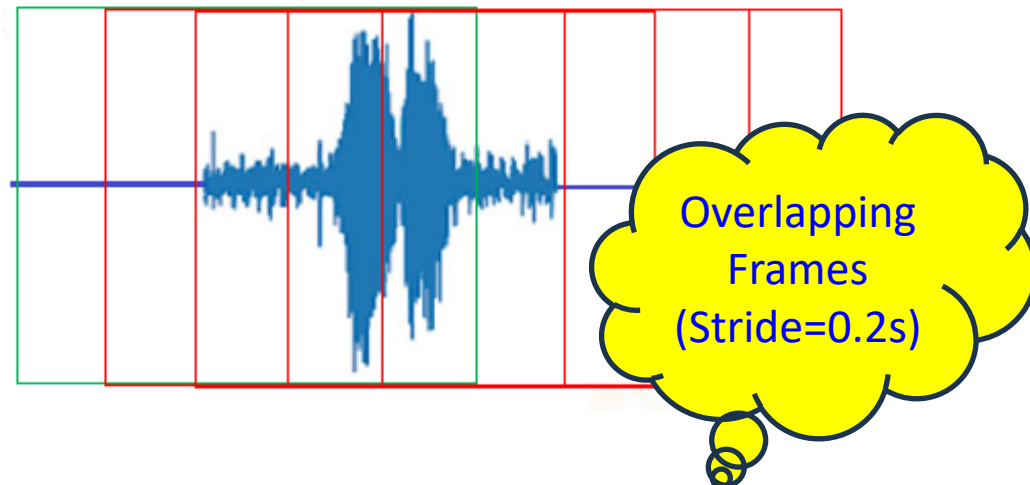
- Inference is run **multiple times per second** to mitigate issue of partial audio capture
 - Consecutive capture frames **overlap** with each other
 - **aggregates** the results of the multiple inferences
 - determine, on **average**, whether a known word is heard



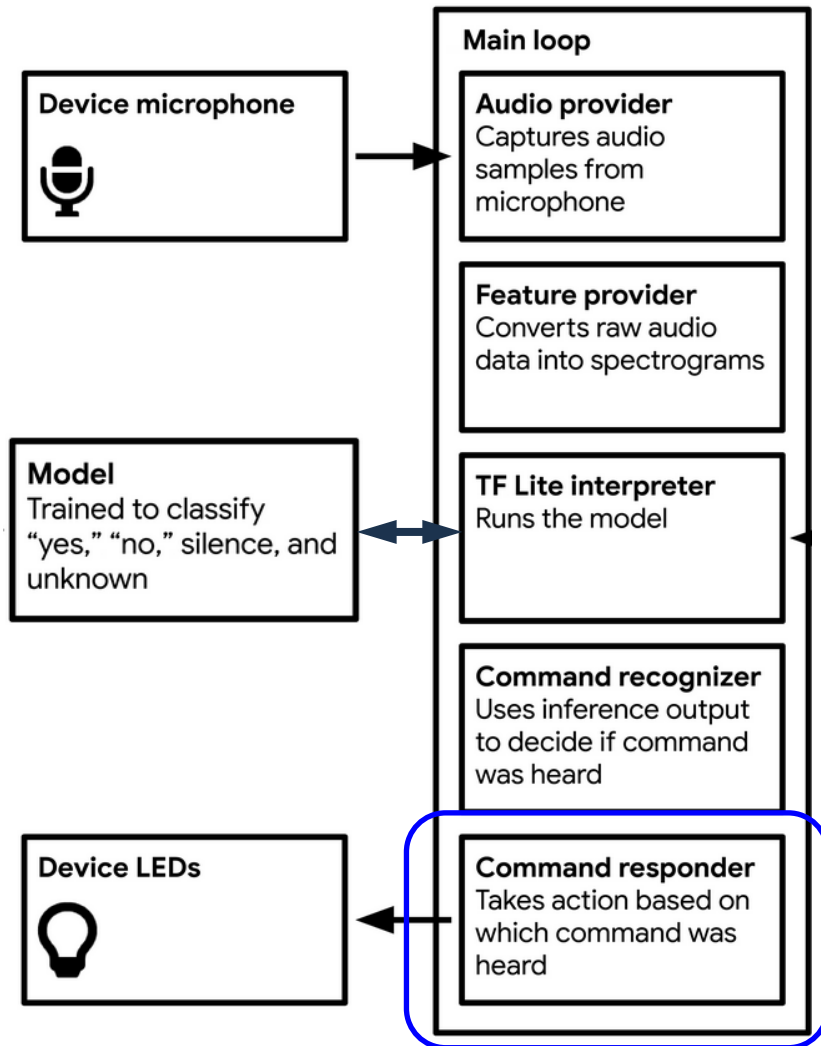
Multiple Inferences



- Audio data for the word 'Noted'
- If only part of the data is captured
 - sounds like 'no'
- E.g. Inference may be performed **every 0.2 seconds** and select the utterance with **highest score/probability**



TinyML Wake Word Detection Example



- **Command responder**
 - response accordingly if a command is heard
- **blink** the appropriate **LED** with appropriate **color** for the application running on the Arduino Nano board



Source: TinyML Pete Warden & Daniel Situnayake

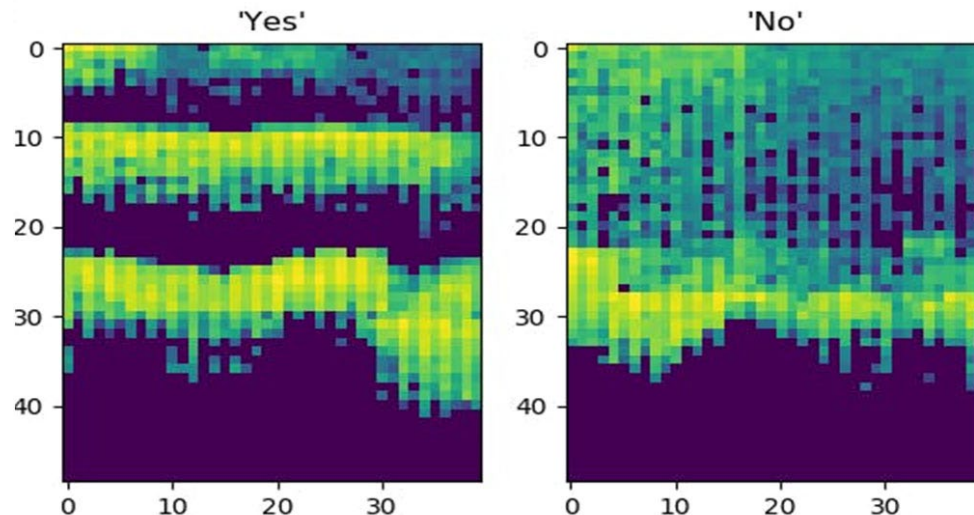
WAKE WORD MODEL TRAINING

TinyML Keyword Detection Application

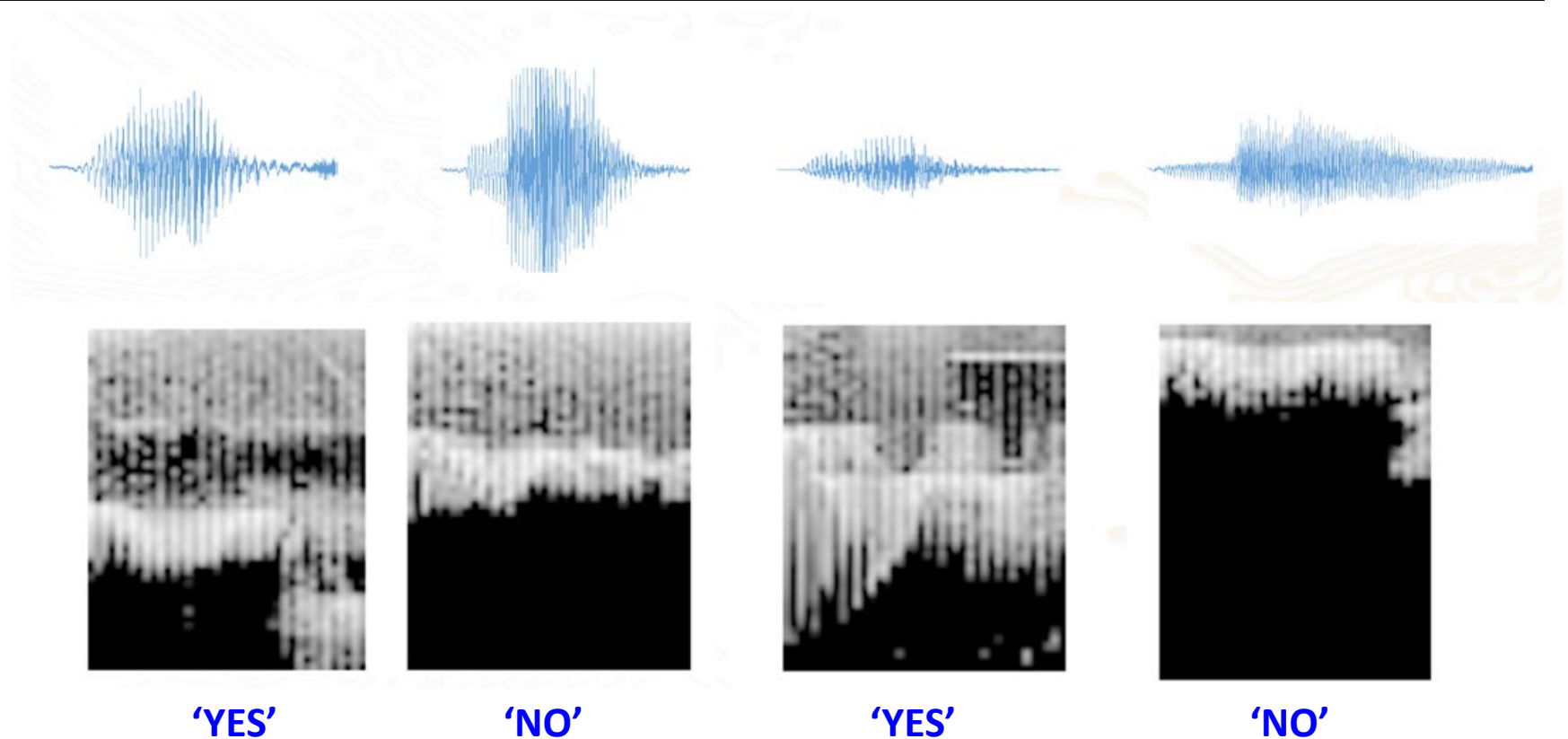
- The **model** (on desktop side) was trained based on a
 - ‘**Speech Commands**’ **dataset**. (From Google).
 - consists of **65,000 one-second-long** utterances of **30** short words crowdsourced online.
- Sample model is trained to only recognize the words
 - “**yes**”
 - “**no**”
 - but is also capable of distinguishing between “**unknown**” words (the **other 28 recorded words**)
 - and “**silence**” or **background noise**.
 - Total **4 classes** of output

TinyML Keyword Detection Operation

- During operation
 - sample 1-second's worth of data at a time
 - outputs four probability scores, one for each of these four classes (yes, no, unknown and silence)
- The sampled audio data is pre-processed and converted into spectrograms



Spectrogram of “yes” and “no”



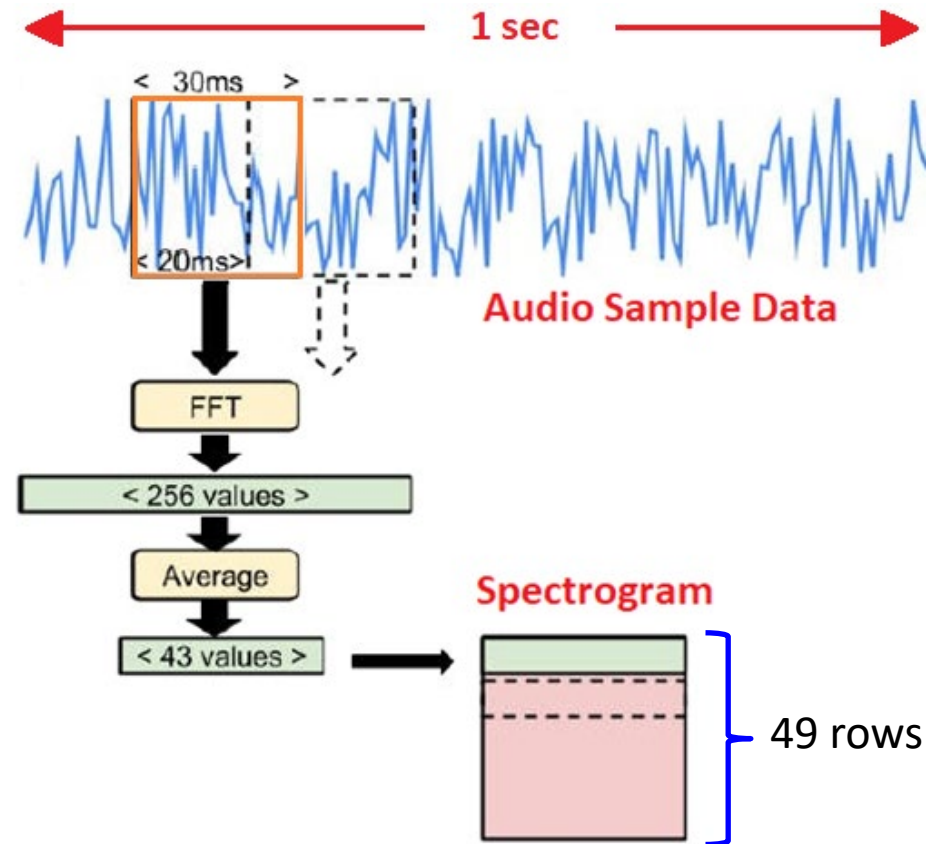
- Spectrograms are also a lot **smaller** than the sample data
 - size = $49 \times 40 = 1960$
- Audio data = 16000 samples/second @ 16KHz sampling rate

Keyword Detection Inference Operation

- Spectrogram is then fed into the model for inference identification
- A spectrogram is
 - Like an image
 - fed into the model as a 2D tensor
- Most effective type of neural network that work well with images such as spectrogram
 - Convolutional Neural Network (CNN)

The Spectrogram

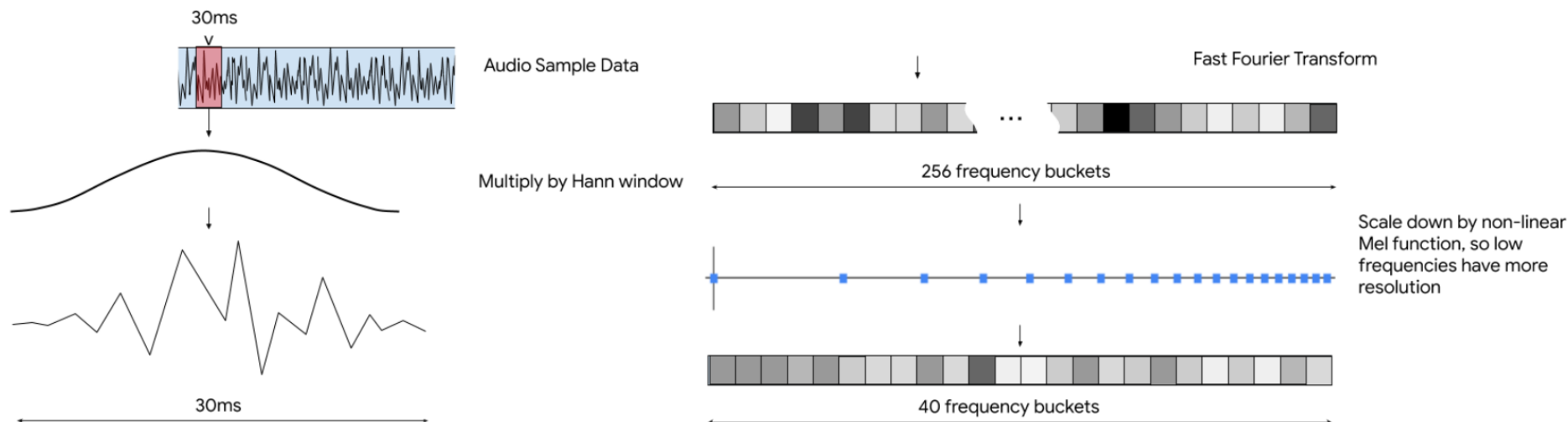
- Each **spectrogram** is represented as a 2D array
 - with **49 rows** and **40 columns**
- Each row represents a **30-millisecond** sample of audio split into **43 frequency buckets**
- **30ms window size** is not **arbitrary**, it is chosen based on the physics of the human vocal tract.
- It takes **20ms to 40ms** for human mouth to **significantly change shape** from one phoneme (like "a") to another (like "o").



Features Extraction

- Every 30msec time slice of data is first filtered with a Hann window
 - a bell-shaped function to reduce the influence of samples at either end of the 30-ms window.
- FFT is then applied at 16K sampling rate
 - 30 msec => 480 samples
 - padded with zeros to 512 samples
 - 256 frequency buckets
 - Average down to 40 frequency buckets using 'Mel' frequency scale which has more weight at lower frequency

The TFLite example uses 43 instead of 40

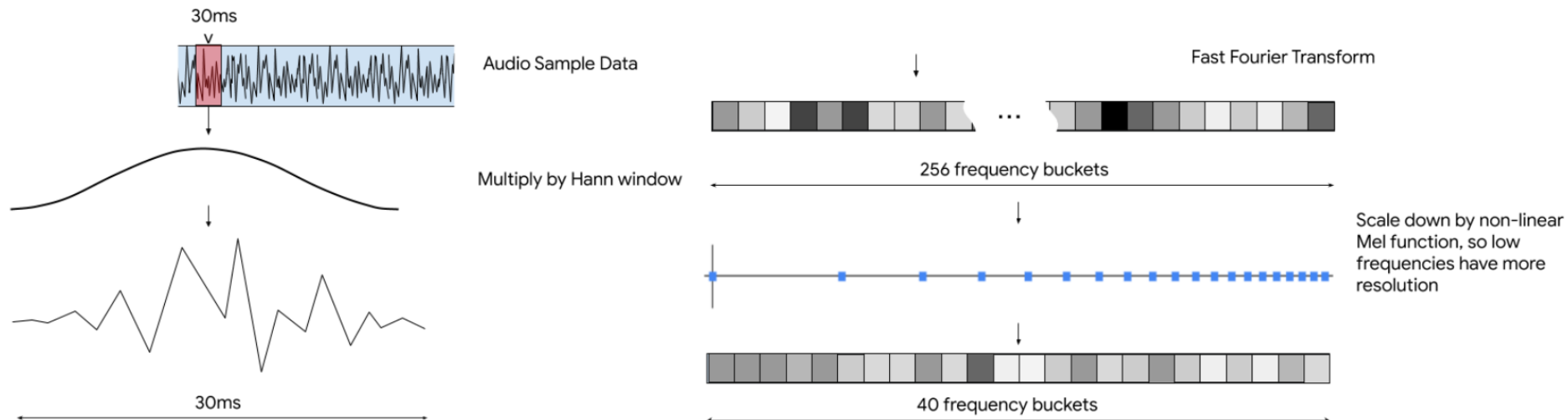


Features Extraction (cont)

Why 49?

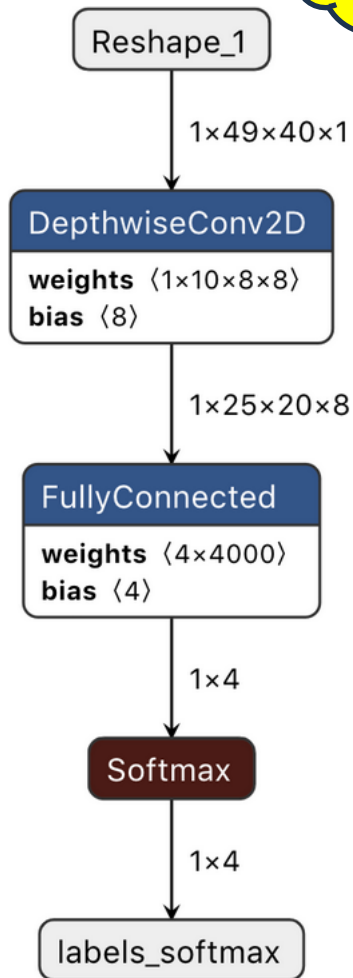
- The process is repeated 49 times per second
 - with a 30-ms window that's moved forward 20 ms each time between iterations
 - cover the full one second of audio input data.
- Output = a 2D array of values that's 40 elements wide
 - one for each frequency bucket and 49 rows high
 - one row for each time slice

Why 40?



Model Architecture

From
Netron



First hidden layer

- a convolution layer (Depthwise Convolution 2D)
- 4D tensors [batch_size, rows, columns, channels]

• 2nd hidden layer

- Fully connected layer

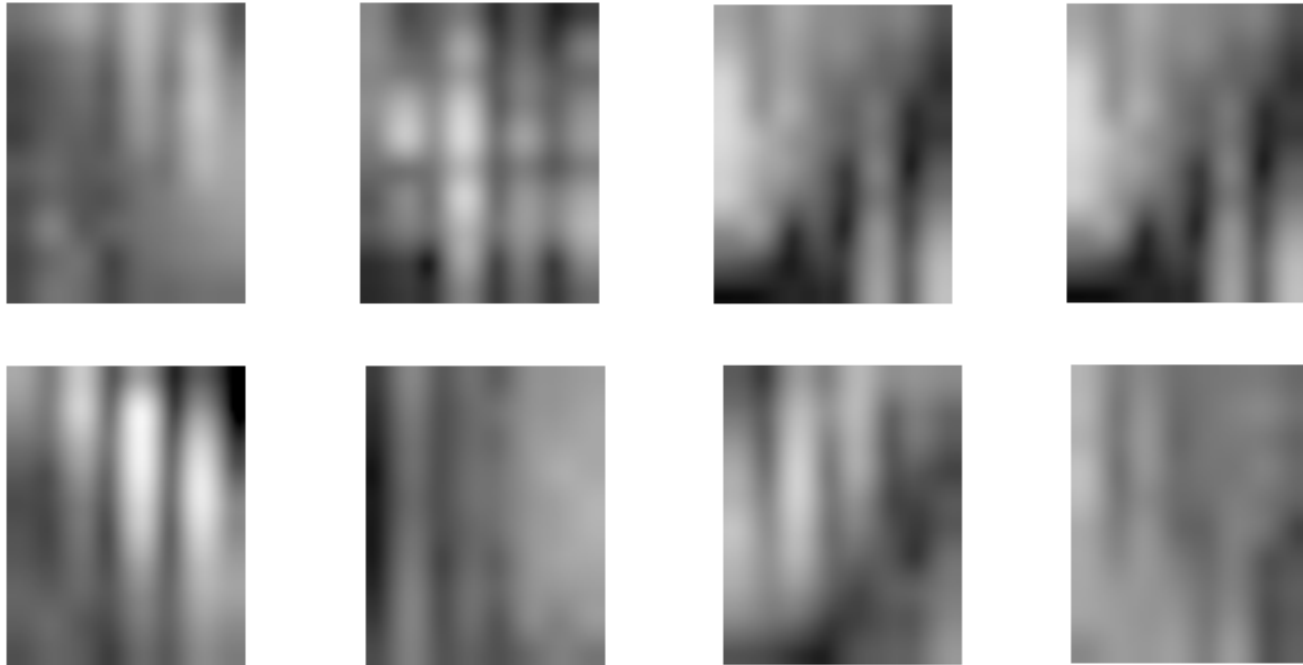
• Output layer

- Softmax activation function
- For classification

```
micro_speech  arduino_audio_provider.cpp  arduino_command_responder.cpp  arduino_main.cpp  a
//
// tflite::AllOpsResolver resolver;
// NOLINTNEXTLINE(runtime-global-variables)
static tflite::MicroMutableOpResolver<4> micro_op_resolver(error_reporter);
if (micro_op_resolver.AddBuiltin(
    tflite::BuiltinOperator_DEPTHWISE_CONV_2D,
    tflite::ops::micro::Register_DEPTHWISE_CONV_2D()) != kTfLiteOk) {
    return;
}
if (micro_op_resolver.AddBuiltin(
    tflite::BuiltinOperator_FULLY_CONNECTED,
    tflite::ops::micro::Register_FULLY_CONNECTED()) != kTfLiteOk) {
    return;
}
if (micro_op_resolver.AddBuiltin(tflite::BuiltinOperator_SOFTMAX,
    tflite::ops::micro::Register_SOFTMAX()) !=
    kTfLiteOk) {
```

The Convolution Filters

- There are 8 filters of size 8x10 pixels used in the convolution layer.



- Each of these filters is slid 2-pixel at a time across the image
- Output will be high when there is a good match between the pattern in the filter and the input image

Convolution Filter operation

More in
CNN
Chapter

- Applying **3x3 filter** to an image consists of 7x7 pixels
- Sliding 2 pixels => output = half the input (3x3)

Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2
0 0 2 2 1 0 0	-1 1 1	
0 2 1 2 0 1 0	-1 0 0	
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 0 0 1 1 0 0		
0 0 0 0 0 0 0		

Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2 0
0 0 2 2 1 0 0	-1 1 1	
0 2 1 2 0 1 0	-1 0 0	
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 0 0 1 1 0 0		
0 0 0 0 0 0 0		

Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2 0 -1
0 0 2 2 1 0 0	-1 1 1	
0 2 1 2 0 1 0	-1 0 0	
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 1 1 1 1 2 0		
0 0 0 0 0 0 0		

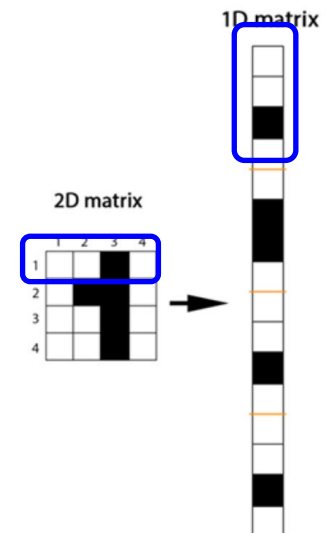
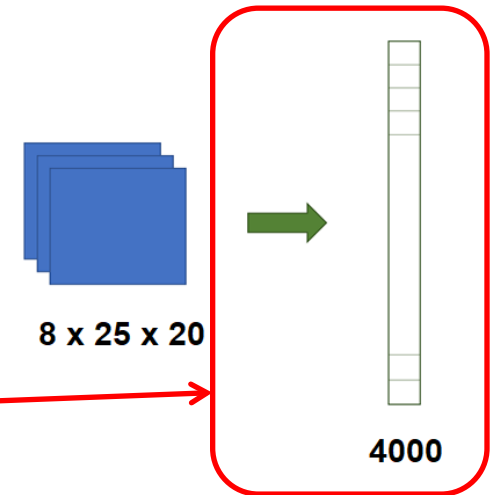
Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2 0 -1
0 0 2 2 1 0 0	-1 1 1	1
0 2 1 2 0 1 0	-1 0 0	
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 1 1 1 1 2 0		
0 0 0 0 0 0 0		

Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2 0 -1
0 0 2 2 1 0 0	-1 1 1	1 0 -1
0 2 1 2 0 1 0	-1 0 0	-1 3
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 0 0 1 1 0 0		
0 0 0 0 0 0 0		

Input	Filter	Output
0 0 0 0 0 0 0	-1 0 -1	2 0 -1
0 0 2 2 1 0 0	-1 1 1	1 0 -1
0 2 1 2 0 1 0	-1 0 0	-1 3 -1
0 1 1 0 1 0 0		
0 2 1 2 0 0 0		
0 1 1 1 1 2 0		
0 0 0 0 0 0 0		

Fully Connected Layer

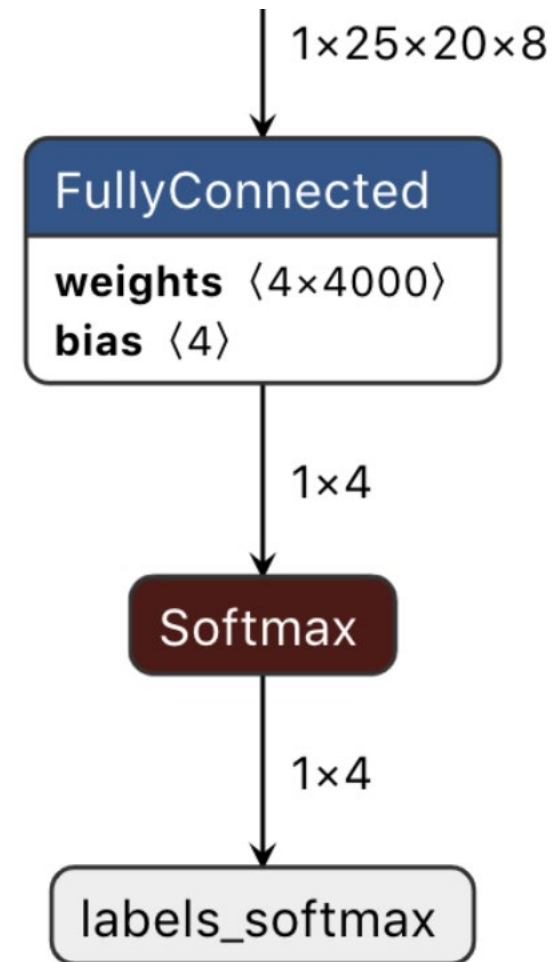
- Output of the Convolution Layer
 - Image Size = 49x40
 - Convolution Filter = 2-pixel slide
 - Output size = 25x20 (half the input)
 - 8 filters (channels)
- Flatten the image and connect to the next layer (Fully connected layer)
 - consists of 4 neurons
 - each with 4000 weights ($=8*25*20$)
 - each outputs the predicted weightage of the four audio



Flattening is typically done after convolution for connection to fully connected layer

Model Output

- Final output layer is a Softmax activation function
 - perform classification
 - “yes”, “no”, “silence”, “unknown”
- Inference is done at about 0.1 second interval (10 to 15 times per seconds)
 - add a post processing function to accumulate and average the scores over time
 - confirm once the score value reach the thresholds



Reference link to Github

Chapter 8 TinyML text

O'REILLY Topics Start Learning Featured

puting resources in the cloud, and it comes set up with too
monitor the training process. It's also completely free.

Over the course of this section, we will use a Colab notebook to train our new model. The notebook we use is available in the TensorFlow repository.

[Open the notebook](#) and click the “Run in Google Colab” button, as shown in [Figure 8-1](#).


[Run in Google Colab](#)


[View source on GitHub](#)

Figure 8-1. The “Run in Google Colab” button

◀ 7. Wake-Word Detection: Building an Applica... 8. Wake-Word Detection: Training a Model 9. Person Detection

7h 26m remaining

